SUBSYSTEMS

# 8-bit Computer Simulator – V3 Addendum

This addendum is meant to supplement the information in the original instruction manual for the previous versions of this device, so it will only address the changes in version 3.

**©2026**

**Subsystems**

# Change Summary

**Version 3 has added these additional features:**

1 – USB-C Power Port (replaces the Micro-USB port).

2 – Mounting holes added to allow for mounting a case or overlay.

3 – A light sensor replaces the temperature sensor.

4 – A partial list of instructions is printed on the back.

5 – Bonus instructions were added (these are not listed on the back).

6 – There is now a 16-byte stack separate from program memory.

7 – There are now 4 slots to store programs (0-3).

# Light Sensor

The light sensor uses the same command number (0x08) as the previous temperature sensor. It loads into the ACCA the current value of the light sensor. This is an analog reading and can extend from 0 to 5 volts. The ADC on the Atmel chip is a 10-bit ADC. Because this is an 8-bit system, the lower (least significant) 2 bits are ignored.

Here is a quick program to use the light sensor and new LBAR command to display a bar graph of the light level.

| Number | ADDR | ASM | Command |
|---|---|---|---|
| 0 | 00 | LIGHT | 08 |
| 1 | 01 | SFLA | 32 |
| 2 | 02 | LBAR | 48 |
| 3 | 03 | JMP | 22 |
| 4 | 04 | | 00 |

This will load the current value of the light sensor into ACCA. It is shifted left once to increase its sensitivity (you can omit this or increase it depending on the light source you are monitoring). We then call LBAR which is a new command that displays a linear bar based on the value of ACCA. Finally, we jump back to the start.

# New Commands

## LIGHT (0x08)

This loads the current value of the light sensor into ACCA. This replaces the temperature command.

## PSHA (0x3F) Push ACCA

You now have a STACK. This is a 16 byte storage for temporary operations. If you PUSH a value from an accumulator, it goes onto the memory stack. The stack pointer is advanced and another value may be pushed. A total of 16 values can be pushed. To get a value from the STACK, you POP it. You can PUSH and POP from either accumulator. But be careful that you don't POP from an empty accumulator or PUSH to a full one. This STACK is external to your memory. Usually, you set a stack pointer to a place in the controller's memory and the stack grows down from there. Since we use the upper memory for certain things, the STACK is a separate memory area so there won't be issues with the stack running into your program. The STACK doesn't track who pushed or popped. When you pop a value it takes the last value pushed. This allows you to push from ACCA and then pop into ACCB. It uses the last in, first out (LIFO) scheme.

## POPA (0x40) Pop ACCA

Pops a value off the stack and transfers it into ACCA.

## PSHB (0x41) Push ACCB

Pushes a value from the ACCB to the STACK.

## POPB (0x42) Pop ACCB

Pops a value off the stack and transfers it into ACCB.

## CALL (0x43) Call a subroutine

You now have the ability to call subroutines. These are sections of code that you may need to run multiple times at different locations in code. When you call a subroutine, the next memory location holds the address of the subroutine you want to run. For instance, if there was a program that required the light level to be checked and some processing to be done frequently, this could be placed in an upper part of memory and you can call it from multiple places in your program. During a subroutine execution if the command RTN is reached, the program execution will move to the next command after the original CALL command was executed. Upon return, the accumulators will be unchanged from any state that they were left in from the subroutine. If you want to ensure you maintain the values of the accumulators prior to calling a subroutine, push their values before the call and then pop them back after you return.

## RTN (0x44) Return from subroutine

During execution of a subroutine, when the RTN command is reached, the program execution will return to the next command after the original CALL command.

## PONG (0x45) Ping Pong graphic on the DATA LEDs

Creating moving lights can take a bunch of code when doing it in machine language, so I am adding a couple of prebuilt ones to make it easier. When executed, the red DATA LEDs will cycle from left to right and back again. You can use this as a simple notification if something triggers it in your program.

## EXPL (0x46) Starburst on both LED rows

This is another graphic where a starburst pattern is visible on both LED rows. This is great if you are making an alarm or if you use it as an end of game signal.

## LDOT (0x47) Linear Dot Graph

This command takes the value of ACCA and displays it as a linear representation on the DATA LEDs. The possible 256 values are broken into 8 buckets and whichever is the highest value bucket is used as the dot to light.

## LBAR (0x48) Linear Bar Graph

This command does the same thing as the LDOT command except it lights all the LEDs up to and including the highest value. This will give you a light bar based on the value of ACCA.

## DLY1 (0x49) Delay 1 Second

Delays are often required in programs and instead of having to go through pushing ACCA, loading it with a value, calling the DLY command and then popping the value back to ACCA, I have included this command to give you a 1 second delay. For many applications this is all that is needed so this should save some time and memory.

| CMD | Binary | ASM | Description | BYTES | CODES |
|---|---|---|---|---|---|
| 0 | **00000000** | **AOUT** | ACCA => DISPLAY | 1 | |
| 1 | **00000001** | **BOUT** | ACCB => DISPLAY | 1 | |
| 2 | **00000010** | **MOUT** | M[REGY] => DISPLAY | 1 | |
| 3 | **00000011** | **INA** | DATA => ACCA (wait for it) | 1 | Z |
| 4 | **00000100** | **KEY** | DATA => ACCA (real-time) | 1 | Z |
| 5 | **00000101** | **TMR** | Timer => ACCA elapsed secs | 1 | Z |
| 6 | **00000110** | **RTMR** | 0 => Timer | 1 | |
| 7 | **00000111** | **BRT** | Set LED brightness (ACCA) | 1 | |
| 8 | **00001000** | **LIGHT** | Light => ACCA | 1 | Z |
| 9 | **00001001** | **LDA** | d => ACCA | 2 | Z |
| 10 | **00001010** | **LDB** | d => ACCB | 2 | Z |
| 11 | **00001011** | **LDY** | d => REGY | 2 | Z |
| 12 | **00001100** | **CLA** | 0 => ACCA | 1 | |
| 13 | **00001101** | **CLB** | 0 => ACCB | 1 | |
| 14 | **00001110** | **INCA** | ACCA + 1 => ACCA | 1 | Z,C |
| 15 | **00001111** | **INCB** | ACCB + 1 => ACCB | 1 | Z,C |
| 16 | **00010000** | **INCY** | REGY + 1 => REGY | 1 | Z,C |
| 17 | **00010001** | **DECA** | ACCA - 1 => ACCA | 1 | Z,N |
| 18 | **00010010** | **DECB** | ACCB - 1 => ACCB | 1 | Z,N |
| 19 | **00010011** | **DECY** | REGY - 1 => REGY | 1 | Z,N |
| 20 | **00010100** | **SWP** | ACCA <=> ACCB | 1 | Z |
| 21 | **00010101** | **SYZ** | REGY <=> REGZ | 1 | Z |
| 22 | **00010110** | **ROLA** | ROLL ACCA 1 BIT LEFT | 1 | Z,C |
| 23 | **00010111** | **RORA** | ROLL ACCA 1 BIT RIGHT | 1 | Z,C |
| 24 | **00011000** | **CMPA** | (ACCA-d); Z if 0, N if <0 | 2 | Z,N |
| 25 | **00011001** | **CMPB** | (ACCB-d); Z if 0, N if <0 | 2 | Z,N |

| | | | | | |
|---|---|---|---|---|---|
| 26 | **00011010** | **CMAB** | (ACCA-ACCB); Z if 0, N if <0 | 2 | Z,N |
| 27 | **00011011** | **MIA** | M[REGY] => ACCA | 1 | Z |
| 28 | **00011100** | **AIM** | ACCA => M[REGY] | 1 | Z |
| 29 | **00011101** | **MIB** | M[REGY] => ACCB | 1 | Z |
| 30 | **00011110** | **BIM** | ACCB => M[REGY] | 1 | Z,N |
| 31 | **00011111** | **AAM** | ACCA+M[REGY] => M[REGY] | 1 | Z,C |
| 32 | **00100000** | **AMA** | ACCA+M[REGY] => ACCA | 1 | Z,C |
| 33 | **00100001** | **SAM** | M[REGY]-ACCA => M[REGY] | 1 | Z,N |
| 34 | **00100010** | **JMP** | d=> PC | 2 | |
| 35 | **00100011** | **WAIT** | Stop until INC pressed | 1 | |
| 36 | **00100100** | **STOP** | Stop program execution | 1 | |
| 37 | **00100101** | **DLYA** | DELAY = ACCA x 0.1 sec | 1 | |
| 38 | **00100110** | **DLYB** | DELAY = ACCB x 0.1 sec | 1 | |
| 39 | **00100111** | **BEQ** | If Z, d => PC | 2 | |
| 40 | **00101000** | **BNE** | If not Z, d => PC | 2 | |
| 41 | **00101001** | **BLT** | If N, d => PC | 2 | |
| 42 | **00101010** | **BLE** | If Z or N, d => PC | 2 | |
| 43 | **00101011** | **BGT** | If not (Z or N), d => PC | 2 | |
| 44 | **00101100** | **BGE** | If not N, d => PC | 2 | |
| 45 | **00101101** | **BCS** | If C, d=>PC | 2 | |
| 46 | **00101110** | **ADDA** | ACCA+d => ACCA | 2 | Z,C |
| 47 | **00101111** | **ADAB** | ACCA+ACCB => ACCA | 1 | Z,C |
| 48 | **00110000** | **SUBA** | ACCA-d => ACCA | 2 | Z,N |
| 49 | **00110001** | **SBA** | ACCA-ACCB => ACCA | 1 | Z,N |
| 50 | **00110010** | **SFLA** | SHIFT LEFT ACCA 1 BIT | 1 | Z,C |
| 51 | **00110011** | **SFLB** | SHIFT LEFT ACCB 1 BIT | 1 | Z,C |
| 52 | **00110100** | **SFRA** | SHIFT RIGHT ACCA 1 BIT | 1 | Z,C |
| 53 | **00110101** | **SFRB** | SHIFT RIGHT ACCB 1 BIT | 1 | Z,C |
| 54 | **00110110** | **ANDA** | ACCA (AND) d => ACCA | 2 | Z |
| 55 | **00110111** | **ORA** | ACCA (OR) d => ACCA | 2 | Z |

| | | | | | |
|---|---|---|---|---|---|
| 56 | **00111000** | **XORA** | ACCA (XOR) d => ACCA | 2 | Z |
| 57 | **00111001** | **AAB** | ACCA (AND) ACCB => ACCA | 1 | Z |
| 58 | **00111010** | **AOB** | ACCA (OR) ACCB => ACCA | 1 | Z |
| 59 | **00111011** | **AXOB** | ACCA (XOR) ACCB => ACCA | 1 | Z |
| 60 | **00111100** | **MULA** | ACCA X d =>M[254,255] | 2 | Z |
| 61 | **00111101** | **MAB** | ACCA X ACCB => M[254,255] | 1 | Z |
| 62 | **00111110** | **RNDA** | RANDOM (0-255) => ACCA | 1 | Z |
| 63 | **00111111** | **PSHA** | ACCA => STACK | 1 | Z |
| 64 | **01000000** | **POPA** | STACK => ACCA | 1 | Z |
| 65 | **01000001** | **PSHB** | ACCB => STACK | 1 | Z |
| 66 | **01000010** | **POPB** | STACK => ACCB | 1 | Z |
| 67 | **01000011** | **CALL** | PC => d; PC+2 => RTN | 2 | |
| 68 | **01000100** | **RTN** | RTN => PC | 1 | |
| 69 | **01000101** | **PONG** | Ping Pong Display | 1 | |
| 70 | **01000110** | **EXPL** | Starburst Display | 1 | |
| 71 | **01000111** | **LDOT** | Linear Dot display of ACCA | 1 | |
| 72 | **01001000** | **LBAR** | Linear Bar Display of ACCA | 1 | |
| 73 | **01001001** | **DLY1** | Delay 1 Second | 1 | |

# Sample programs

## Light Alarm

This program will wait 4 seconds then load a reference light level. If the light level goes above or below the reference plus or minus 2 units, the EXPL animation will be shown.

| Address | ASM | Machine Code |
|---|---|---|
| 00000000 | LDA | 09 |
| 00000001 | | 40 |
| 00000010 | DLYA | 25 |
| 00000011 | LIGHT | 08 |
| 00000100 | SWP | 14 |
| 00000101 | PSHB | 41 |
| 00000110 | INCB | 0F |
| 00000111 | INCB | 0F |
| 00001000 | LIGHT | 08 |
| 00001001 | AO | 00 |
| 00001010 | CMAB | 1A |
| 00001011 | BGT | 2B |
| 00001100 | | 17 |
| 00001101 | POPB | 42 |
| 00001110 | PSHB | 41 |
| 00001111 | DECB | 12 |
| 00010000 | DECB | 12 |
| 00010001 | CMAB | 1A |

| | | |
|---|---|---|
| 00010010 | BLT | 29 |
| 00010011 | | 17 |
| 00010100 | POPB | 42 |
| 00010101 | JMP | 22 |
| 00010110 | | 05 |
| 00010111 | EXPL | 46 |
| 00011000 | STOP | 24 |

## SLOTS

In this game, which uses the CALL and RTN instructions, you will see 3 groups of two-bit numbers. They will cycle at lowering speed until they lock in. Then you can see if the 3 numbers match. The WAIT command will then wait for you to press the RUN key to start another sequence. You may want to add a section of code to do an EXPL display if you get three of the same.

| Address | ASM | Machine Code |
|---|---|---|
| 00000000 | LDB | 00001010 |
| 00000001 | | 00000001 |
| 00000010 | CALL | 01000011 |
| 00000011 | | 00010000 |
| 00000100 | AO | 00000000 |
| 00000101 | DLYB | 00100110 |
| 00000110 | SFLB | 00110011 |
| 00000111 | CMPB | 00011001 |
| 00001000 | | 00100000 |
| 00001001 | BEQ | 00100111 |

| | | |
|---|---|---|
| 00001010 | | 00001101 |
| 00001011 | JMP | 00100010 |
| 00001100 | | 00000010 |
| 00001101 | WAIT | 00100011 |
| 00001110 | JMP | 00100010 |
| 00001111 | | 00000000 |
| 00010000 | RNDA | 00111110 |
| 00010001 | ANDA | 00101110 |
| 00010010 | | 11011011 |
| 00010011 | RTN | 01000100 |